```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:camera/camera.dart';
import 'package:google_mlkit_text_recognition/google_mlkit_text_recognition.dart';
import 'package:shared_preferences/shared_preferences.dart';
import 'package:image_picker/image_picker.dart';

late List<CameraDescription> cameras;

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  try {
    cameras = await availableCameras();
  } on CameraException catch (e) {
    debugPrint('Error initializing cameras: $e');
    cameras = [];
  }
  runApp(const AusPostApp());
}

class AusPostApp extends StatelessWidget {
  const AusPostApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
```

```
    debugShowCheckedModeBanner: false,
    title: 'AusPost Round Assistant',
    theme: ThemeData.dark().copyWith(
     primaryColor: Colors.redAccent,
     scaffoldBackgroundColor: const Color(0xFF121212),
     appBarTheme: const AppBarTheme(
      backgroundColor: Color(0xFF1F1F1F),
      elevation: 0,
     ),
    ),
    home: const MainNavigationScreen(),
   );
 }
}

//
================================================================================
===
// MAIN NAVIGATION WITH TABS
//
================================================================================
===
class MainNavigationScreen extends StatefulWidget {
 const MainNavigationScreen({super.key});

 @override
 State<MainNavigationScreen> createState() => _MainNavigationScreenState();
```

```
}

class _MainNavigationScreenState extends State<MainNavigationScreen> {
 int _currentIndex = 0;
 Map<String, String> _streetRoundData = {};
 bool _isLoading = true;

 @override
 void initState() {
  super.initState();
  _loadData();
 }

 // Load streets from local SharedPreferences phone storage
 Future<void> _loadData() async {
  final prefs = await SharedPreferences.getInstance();
  String? rawJson = prefs.getString('streets_db');
  if (rawJson != null) {
   Map<String, dynamic> decoded = json.decode(rawJson);
   setState(() {
    _streetRoundData = decoded.map((k, v) => MapEntry(k, v.toString()));
    _isLoading = false;
   });
  } else {
   // Default sample data if fresh install
   Map<String, String> initialData = {
```

```
    "HIGH STREET": "ROUND 01",
    "STATION ROAD": "ROUND 05",
    "VICTORIA PARADE": "ROUND 12",
   };
   await _saveData(initialData);
   setState(() {
    _isLoading = false;
   });
  }
}

// Save updated streets to local phone storage
Future<void> _saveData(Map<String, String> data) async {
  final prefs = await SharedPreferences.getInstance();
  await prefs.setString('streets_db', json.encode(data));
  setState(() {
   _streetRoundData = data;
  });
}

void _addOrUpdateStreet(String street, String round) {
  Map<String, String> updated = Map.from(_streetRoundData);
  updated[street.trim().toUpperCase()] = round.trim().toUpperCase();
  _saveData(updated);
}
```

```
void _deleteStreet(String street) {
  Map<String, String> updated = Map.from(_streetRoundData);
  updated.remove(street);
  _saveData(updated);
}

@override
Widget build(BuildContext context) {
  if (_isLoading) {
    return const Scaffold(
      body: Center(child: CircularProgressIndicator(color: Colors.redAccent)),
    );
  }

  final pages = [
    ScannerTab(streetData: _streetRoundData),
    StreetListTab(
      streetData: _streetRoundData,
      onAddOrUpdate: _addOrUpdateStreet,
      onDelete: _deleteStreet,
    ),
  ];

  return Scaffold(
    body: pages[_currentIndex],
    bottomNavigationBar: BottomNavigationBar(
```

```
      currentIndex: _currentIndex,
      selectedItemColor: Colors.redAccent,
      unselectedItemColor: Colors.grey,
      backgroundColor: const Color(0xFF1F1F1F),
      onTap: (index) => setState(() => _currentIndex = index),
      items: const [
       BottomNavigationBarItem(
        icon: Icon(Icons.qr_code_scanner),
        label: 'Live Scan',
       ),
       BottomNavigationBarItem(
        icon: Icon(Icons.list_alt),
        label: 'Manage Streets',
       ),
      ],
     ),
    );
  }
}

//
================================================================================
===
// TAB 1: LIVE SCANNER
//
================================================================================
===
```

```
class ScannerTab extends StatefulWidget {
  final Map<String, String> streetData;
  const ScannerTab({super.key, required this.streetData});

  @override
  State<ScannerTab> createState() => _ScannerTabState();
}

class _ScannerTabState extends State<ScannerTab> {
  CameraController? _cameraController;
  final TextRecognizer _textRecognizer = TextRecognizer();
  bool _isProcessing = false;

  String _detectedStreet = "Point camera at address";
  String _detectedRound = "--";

  @override
  void initState() {
    super.initState();
    _initCamera();
  }

  void _initCamera() {
    if (cameras.isEmpty) return;

    _cameraController = CameraController(
```

```
    cameras[0],
    ResolutionPreset.medium,
    enableAudio: false,
  );

  _cameraController!.initialize().then((_) {
    if (!mounted) return;
    // Start processing frames
    _cameraController!.startImageStream((CameraImage image) {
      if (!_isProcessing) {
        _isProcessing = true;
        _matchStreetDataLive(image);
      }
    });
    setState(() {});
  }).catchError((e) {
    debugPrint('Error initializing camera controller: $e');
  });
}

// NOTE: True live text recognition from CameraImage stream is complex in Flutter.
// We simulate it here, but typically you'd use a package like google_mlkit_commons
// to convert CameraImage to InputImage.
// For this app, the snap photo fallback works best for text recognition.
Future<void> _matchStreetDataLive(CameraImage image) async {
  // Basic delay simulation for live stream OCR
```

```
  await Future.delayed(const Duration(milliseconds: 1000));

  if (mounted) {
    setState(() {
      _isProcessing = false;
    });
  }
}

// Fallback scanner modal for photos of mail
Future<void> _scanMailPhoto() async {
  final picker = ImagePicker();
  final image = await picker.pickImage(source: ImageSource.camera);
  if (image == null) return;

  // Show loading indicator
  setState(() {
    _detectedStreet = "Scanning...";
    _detectedRound = "--";
  });

  final inputImage = InputImage.fromFilePath(image.path);
  final RecognizedText recognizedText =
      await _textRecognizer.processImage(inputImage);

  bool found = false;
```

```
// Look for street names in the recognized text
for (String blockText in recognizedText.blocks.map((b) => b.text)) {
  String upper = blockText.toUpperCase();

  // Check each street in our database against the scanned text
  for (var entry in widget.streetData.entries) {
    // If the scanned text contains our street name
    if (upper.contains(entry.key)) {
      setState(() {
        _detectedStreet = entry.key;
        _detectedRound = entry.value;
      });
      found = true;
      break;
    }
  }
  if (found) break;
}

if (!found) {
  setState(() {
    _detectedStreet = "Not found";
    _detectedRound = "--";
  });
  if (mounted) {
    ScaffoldMessenger.of(context).showSnackBar(
```

```
      const SnackBar(
       content: Text('No matching street found in system.'),
       backgroundColor: Colors.redAccent,
      ),
     );
   }
 }
}

@override
void dispose() {
 _cameraController?.dispose();
 _textRecognizer.close();
 super.dispose();
}

@override
Widget build(BuildContext context) {
 if (cameras.isEmpty) {
   return Scaffold(
    appBar: AppBar(title: const Text('AusPost Mail Scanner')),
    body: Center(
     child: Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
       const Text('No camera available.'),
```

```
          const SizedBox(height: 20),
          ElevatedButton.icon(
            icon: const Icon(Icons.add_a_photo),
            label: const Text('Scan from Photo'),
            onPressed: _scanMailPhoto,
          )
        ],
      ),
    ),
  );
}

if (_cameraController == null || !_cameraController!.value.isInitialized) {
  return const Scaffold(body: Center(child: CircularProgressIndicator(color: Colors.redAccent)));
}

return Scaffold(
  appBar: AppBar(
    title: const Text('AusPost Mail Scanner'),
    actions: [
      IconButton(
        icon: const Icon(Icons.add_a_photo),
        tooltip: 'Snap Parcel Photo',
        onPressed: _scanMailPhoto,
      )
```

```
  ],
),
body: Stack(
  children: [
    // Camera feed fills the screen
    SizedBox(
      width: double.infinity,
      height: double.infinity,
      child: CameraPreview(_cameraController!),
    ),

    // Overlay Target Box
    Center(
      child: Container(
        width: 320,
        height: 160,
        decoration: BoxDecoration(
          border: Border.all(color: Colors.redAccent, width: 3),
          borderRadius: BorderRadius.circular(16),
        ),
        child: const Align(
          alignment: Alignment.topCenter,
          child: Padding(
            padding: EdgeInsets.all(8.0),
            child: Text(
              'Align address here\nTap camera icon to scan',
```

```
        textAlign: TextAlign.center,
        style: TextStyle(
          color: Colors.white,
          backgroundColor: Colors.black54,
        ),
      ),
    ),
  ),
 ),
),

// Result Display Card
Positioned(
 bottom: 25,
 left: 20,
 right: 20,
 child: Container(
  padding: const EdgeInsets.all(20),
  decoration: BoxDecoration(
   color: Colors.black.withOpacity(0.9),
   borderRadius: BorderRadius.circular(16),
   border: Border.all(color: Colors.redAccent.withOpacity(0.5)),
  ),
  child: Column(
   mainAxisSize: MainAxisSize.min,
   children: [
```

```
            const Text("ROUND NUMBER",
              style: TextStyle(color: Colors.grey, fontSize: 12)),
            Text(
             _detectedRound,
             style: const TextStyle(
               fontSize: 40,
               fontWeight: FontWeight.bold,
               color: Colors.greenAccent,
             ),
            ),
            const SizedBox(height: 5),
            Text(
             _detectedStreet,
             style: const TextStyle(
                fontSize: 20,
                fontWeight: FontWeight.w600,
                color: Colors.white),
             textAlign: TextAlign.center,
            ),
          ],
        ),
      ),
    )
  ],
 ),
);
```

```
  }
}

//
================================================================================
===
// TAB 2: MANAGE STREETS (ADD, EDIT, DELETE, OCR DOCUMENT SCAN)
//
================================================================================
===
class StreetListTab extends StatefulWidget {
  final Map<String, String> streetData;
  final Function(String, String) onAddOrUpdate;
  final Function(String) onDelete;

  const StreetListTab({
    super.key,
    required this.streetData,
    required this.onAddOrUpdate,
    required this.onDelete,
  });

  @override
  State<StreetListTab> createState() => _StreetListTabState();
}

class _StreetListTabState extends State<StreetListTab> {
```

```
TextEditingController searchController = TextEditingController();
String filterText = "";

// Open dialog to add or edit street
void _showStreetDialog({String? initialStreet, String? initialRound}) {
 final streetCtrl = TextEditingController(text: initialStreet ?? "");
 final roundCtrl = TextEditingController(text: initialRound ?? "");

 showDialog(
  context: context,
  builder: (context) => AlertDialog(
   title: Text(initialStreet == null ? 'Add New Street' : 'Edit Round'),
   content: Column(
    mainAxisSize: MainAxisSize.min,
    children: [
     TextField(
      controller: streetCtrl,
      textCapitalization: TextCapitalization.characters,
      decoration: const InputDecoration(
       labelText: 'Street Name (e.g. HIGH ST)',
      ),
      enabled: initialStreet == null, // Lock street name if editing
     ),
     const SizedBox(height: 10),
     TextField(
      controller: roundCtrl,
```

```
          textCapitalization: TextCapitalization.characters,
          decoration: const InputDecoration(
            labelText: 'Round Number (e.g. ROUND 04)',
          ),
        ),
      ],
    ),
    actions: [
      TextButton(
        onPressed: () => Navigator.pop(context),
        child: const Text('Cancel'),
      ),
      ElevatedButton(
        style: ElevatedButton.styleFrom(backgroundColor: Colors.redAccent),
        onPressed: () {
          if (streetCtrl.text.isNotEmpty && roundCtrl.text.isNotEmpty) {
            widget.onAddOrUpdate(streetCtrl.text, roundCtrl.text);
            Navigator.pop(context);
          }
        },
        child: const Text('Save'),
      )
    ],
  ),
 );
}
```

```
  // Scan paper sheet/document with phone camera to auto-extract streets & rounds
  Future<void> _scanPaperDocument() async {
    final picker = ImagePicker();
    final image = await picker.pickImage(source: ImageSource.camera);
    if (image == null) return;

    // Show loading indicator
    if (!mounted) return;
    showDialog(
      context: context,
      barrierDismissible: false,
      builder: (context) => const Center(child: CircularProgressIndicator(color:
Colors.redAccent)),
    );

    final inputImage = InputImage.fromFilePath(image.path);
    final textRecognizer = TextRecognizer();
    final RecognizedText recognizedText =
        await textRecognizer.processImage(inputImage);

    textRecognizer.close();

    // Close loading indicator
    if (!mounted) return;
    Navigator.pop(context);
```

```
// Show recognized text dialog for fast import
showDialog(
  context: context,
  builder: (context) => AlertDialog(
    title: const Text('Scanned Sheet Text'),
    content: SizedBox(
      width: double.maxFinite,
      child: SingleChildScrollView(
        child: SelectableText(
          recognizedText.text.isEmpty
              ? 'No readable text found.'
              : recognizedText.text,
        ),
      ),
    ),
    actions: [
      TextButton(
        onPressed: () => Navigator.pop(context),
        child: const Text('Close'),
      ),
      ElevatedButton(
        style: ElevatedButton.styleFrom(backgroundColor: Colors.redAccent),
        onPressed: () {
          Navigator.pop(context);
          _showStreetDialog();
```

```
        },
        child: const Text('Add Street Manually'),
       )
     ],
    ),
  );
}

@override
Widget build(BuildContext context) {
  // Sort alphabetically and filter
  var sortedEntries = widget.streetData.entries.toList()
    ..sort((a, b) => a.key.compareTo(b.key));

  final filteredList = sortedEntries.where((e) {
    return e.key.contains(filterText.toUpperCase()) ||
        e.value.contains(filterText.toUpperCase());
  }).toList();

  return Scaffold(
    appBar: AppBar(
      title: const Text('Street & Round Database'),
      actions: [
        IconButton(
          icon: const Icon(Icons.document_scanner),
          tooltip: 'Scan Paper List Sheet',
```

```
    onPressed: _scanPaperDocument,
   )
  ],
 ),
 floatingActionButton: FloatingActionButton.extended(
  backgroundColor: Colors.redAccent,
  icon: const Icon(Icons.add, color: Colors.white),
  label: const Text('Add Street', style: TextStyle(color: Colors.white)),
  onPressed: () => _showStreetDialog(),
 ),
 body: Column(
  children: [
   // Search Box
   Padding(
    padding: const EdgeInsets.all(12.0),
    child: TextField(
     controller: searchController,
     decoration: InputDecoration(
      hintText: 'Search street name or round...',
      prefixIcon: const Icon(Icons.search),
      border: OutlineInputBorder(
       borderRadius: BorderRadius.circular(12),
       borderSide: BorderSide.none,
      ),
      filled: true,
      fillColor: const Color(0xFF2A2A2A),
```

```
    suffixIcon: filterText.isNotEmpty
        ? IconButton(
          icon: const Icon(Icons.clear),
          onPressed: () {
            searchController.clear();
            setState(() => filterText = "");
          },
        )
        : null,
    ),
    onChanged: (val) => setState(() => filterText = val),
  ),
),
// List of Streets
Expanded(
  child: filteredList.isEmpty
      ? const Center(child: Text('No streets found. Add some!'))
      : ListView.builder(
          itemCount: filteredList.length,
          itemBuilder: (context, index) {
            final item = filteredList[index];
            return Card(
              margin: const EdgeInsets.symmetric(
                  horizontal: 12, vertical: 4),
              color: const Color(0xFF2A2A2A),
              shape: RoundedRectangleBorder(
```

```
 borderRadius: BorderRadius.circular(10),
),
child: ListTile(
 title: Text(
  item.key,
  style: const TextStyle(
    fontWeight: FontWeight.bold,
    fontSize: 16),
 ),
 subtitle: Text(
  item.value,
  style: const TextStyle(
    color: Colors.greenAccent,
    fontWeight: FontWeight.w600,
    fontSize: 14),
 ),
 trailing: Row(
  mainAxisSize: MainAxisSize.min,
  children: [
   IconButton(
    icon: const Icon(Icons.edit,
      color: Colors.blueAccent),
    onPressed: () => _showStreetDialog(
     initialStreet: item.key,
     initialRound: item.value,
    ),
```

```
),
IconButton(
 icon: const Icon(Icons.delete,
    color: Colors.redAccent),
 onPressed: () {
  showDialog(
   context: context,
   builder: (context) => AlertDialog(
    title: const Text('Delete Street?'),
    content: Text('Are you sure you want to delete ${item.key}?'),
    actions: [
     TextButton(
      onPressed: () => Navigator.pop(context),
      child: const Text('Cancel'),
     ),
     TextButton(
      onPressed: () {
       widget.onDelete(item.key);
       Navigator.pop(context);
      },
      child: const Text('Delete', style: TextStyle(color: Colors.redAccent)),
     ),
    ],
   )
  );
 },
```

```
                    ),
                ],
              ),
            ),
          );
        },
      ),
    ),
  ],
 ),
 );
 }
}
```